

COMP24112 Summative Exercise On Linear Classification and MLP Regression

Student Anthony Nguyen

Loan approval prediction by linear classification

1. Summarize (1) your process of constructing the loss function and computing its gradient used by 'linear train function', (2) how your Python implementation minimizes this function, and (3) key insights learned from the task. (*3 marks*) 2. Draw conclusions about your model behaviour and data from your plot produced in Section 1.2 based on classification accuracies of your training and testing sets? (*1 mark*) 3. Discuss the effect of learning rate on model training and on testing performance, based on your observations obtained in Section 1.3. (*2 marks*)

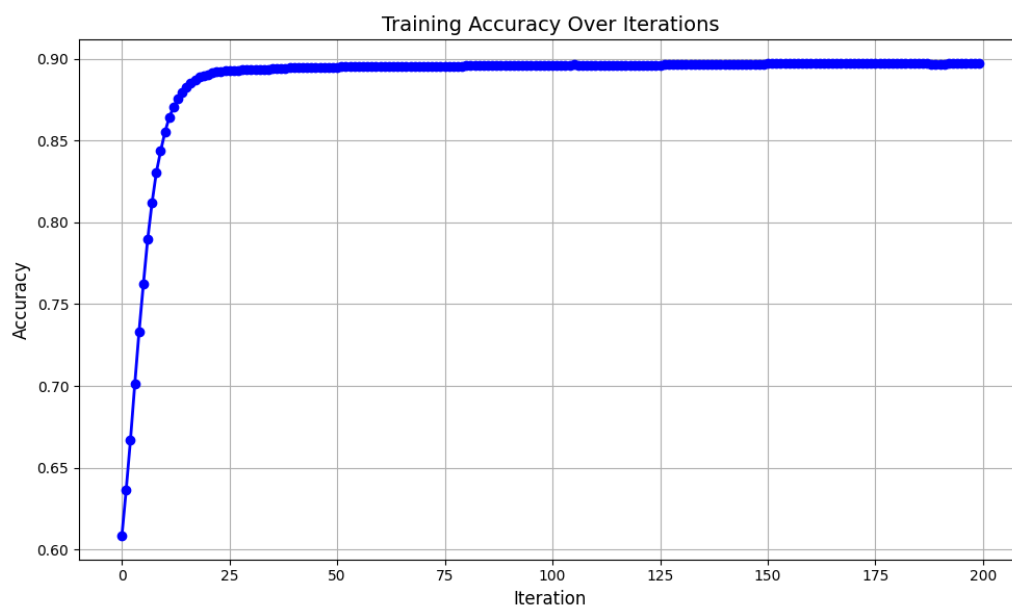
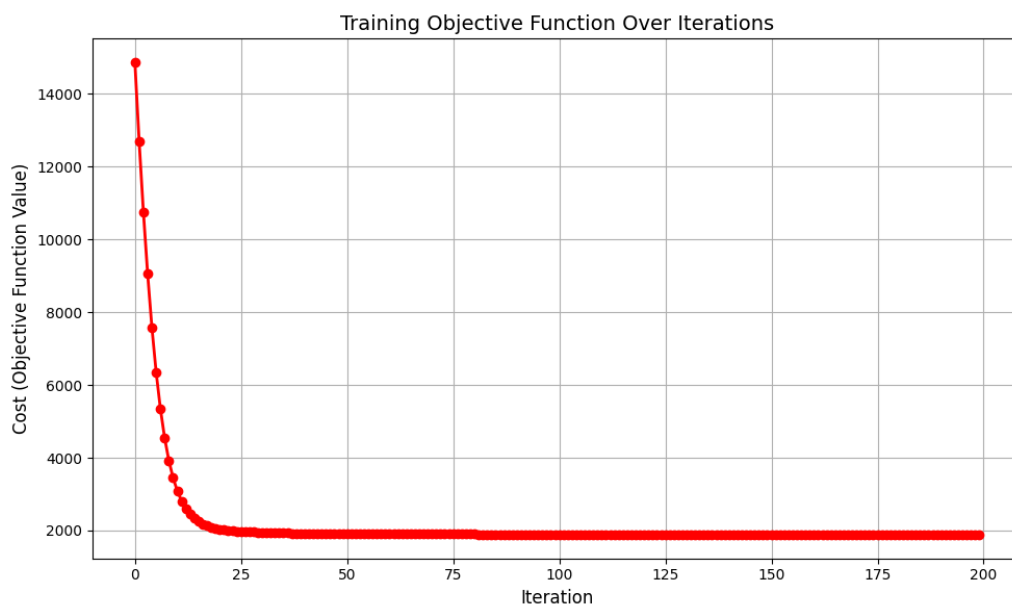
0.1 Constructing the loss function

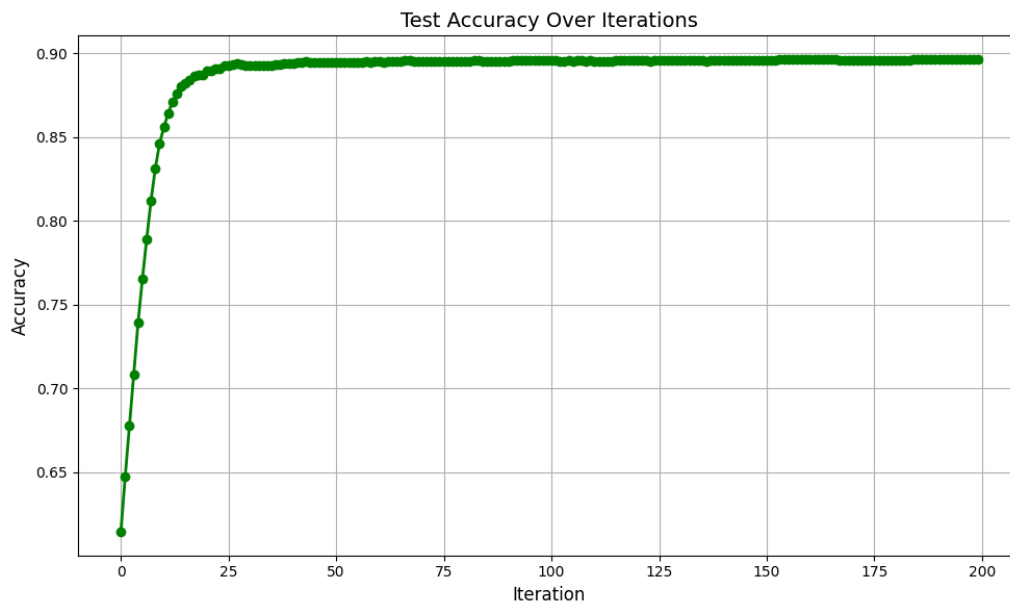
- The loss function was constructed by first adding a bias in the data set (by adding a column of ones) to shift the decision boundary away from the origin for a better representation of the data. I then created a column vector of hinge losses for all samples, and using this vector, summed them and applied the appropriate regularization terms (to avoid overfitting) for the total cost of the model.
- I then computed the gradient by covering two cases, one case where there is no loss and the model predicts correctly where the gradient is 0 and two when the model has a misprediction and the gradient is the derivative of the cost function with respect to w (the weights matrix). The overall function was calculated using matrix operations for efficiency. I then minimized this function by applying gradient descent to a range of learning rates, updating the weights iteratively as the gradient converged to a 0.
- Key insights that I learned from the task would be that the model performs best with a learning rate of 0.0001 as by using plots showed that this had the greatest training and test accuracy as well as the highest F1 score.

0.2 Drawing conclusions

By observing the plot of the training objective function we can see clearly that the graph has an initial high cost of 16000 and it steeply decreases in the first 20 iteration and begins to plateau at 2000. This is expected behavior as the gradient of the loss function converges to 0 as at this point our loss function will be producing its minimum value and thus cost is minimal.

The accuracy against iteration for test and training data plots show a steep increase of accuracy in the first 20 iterations which then plateaus at 90 percent. This is expected as although the weights get better at classifying with more iterations there are diminishing returns due to the model needing to learn more subtle and complex features. Diminishing returns could also be due to noise, data quality or limited information.

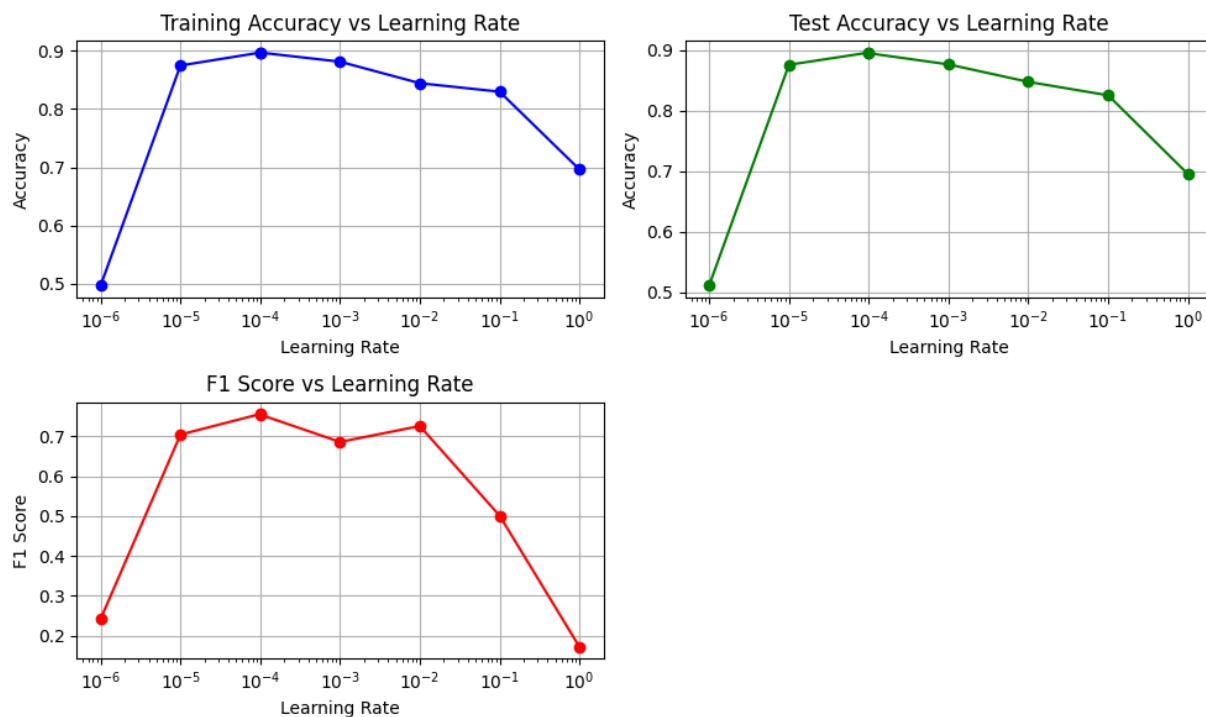




0.3 Effect of learning rate

Observing the plots of training, test, and F1 score at different learning rates we can see that the most optimal learning rate is 0.0001. From this value as we decrease the learning rate our accuracy of both test and accuracy data decrease and we observe the same behaviour when we increase the learning rate from this value.

As the learning rate increases beyond the optimal value, the F1 score generally decreases but exhibits fluctuations, never exceeding the peak performance achieved at the optimal learning rate. This behaviour suggests that higher learning rates lead to unstable training, causing the model to explore different precision-recall trade-offs. The F1 score, being the harmonic mean of precision and recall, reflects these varying trade-offs in a non-monotonic manner. Additionally, the class imbalance (78 percent rejected loans) might exacerbate this instability, making the model overly sensitive to the minority class at higher learning rates..



Soybean Production Prediction by MLP

1. Draw conclusions from your model selection results obtained in Section 2.2, based on factors like prediction accuracy, training efficiency, and model complexity. (*2 marks*)
2. Describe the design process of your selection and validation methods used in Section 2.3, and discuss your observations from your validation experiment. (*2 marks*)
3. Describe the methods you have used to address missing and noisy features in Section 2.4. (*1 mark*)

0.4 Drawing Conclusions from model selection

- **High Prediction Accuracy:** The best Multi-Layer Perceptron (MLP) model achieved a very low mean square error (MSE) of approximately 2.01 and a very high R-squared value of around 0.999999 on the testing set. This means that the selected model has a high ability to predict 'Seed Yield per Unit Area (SUYA)' and explains almost all variance in the target variable.
- **Training Efficiency:** While the total training time was approximately 821.69 seconds this includes the hyper parameter search across 120 different model configurations (24 parameter combinations across 5 cross-validation folds). The early stopping mechanisms within the MLP likely contributed to preventing excessive training time for individual models when the loss stopped improving. The use of the 'adam' solver and optimized batch size also likely contributed to efficient convergence. By using an automatic batch size it uses $\min(200, \text{number of samples})$ to balance training speed and generalization and using Adam (Adaptive Moment Estimation) optimises training by leveraging momentum (Helps accelerate SGD by navigating faster through shallow regions or plateaus) and RMSProp (Adjusts learning rates for each parameter individually based on recent gradient magnitudes)
- **Model Complexity:** The selected model with two hidden layers of 100 neurons each portrays a moderately complex neural network. While capable of learning intricate relationships it isn't excessively deep or wide which helps in balancing model capacity with the risk of over fitting. The effectiveness of this complexity is evident in the high prediction accuracy achieved.

In summary, the model selection process identified a well-performing MLP architecture with optimized hyper parameters that demonstrates excellent prediction accuracy on the test data without excessive training time, suggesting a good balance between model complexity and generalization ability.

0.5 Design process

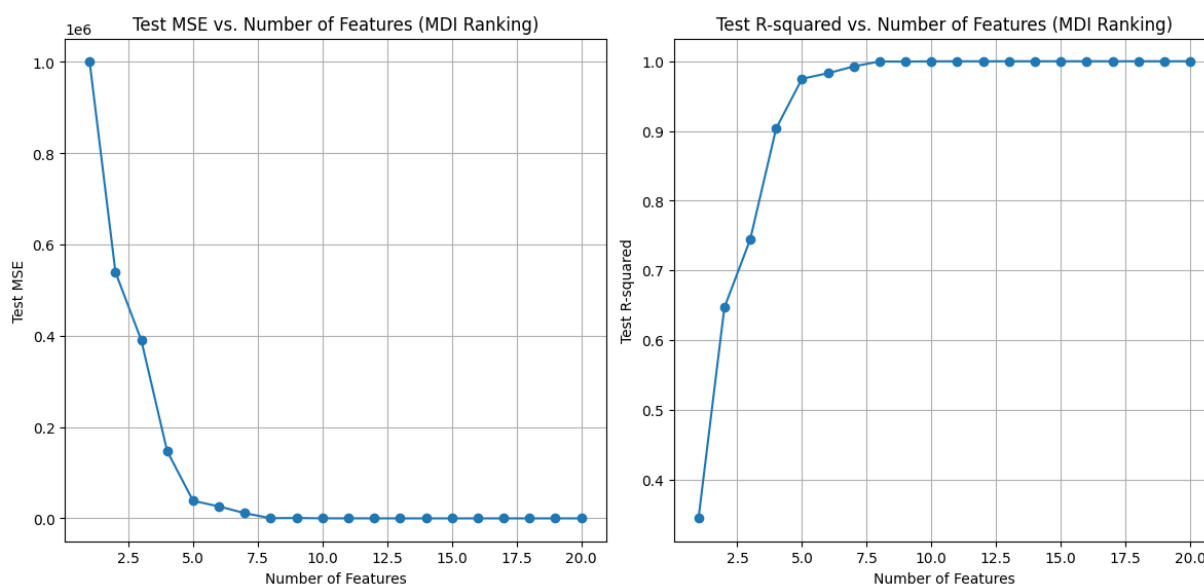
- **Feature Ranking:** The calculated feature importance scores were then used to rank the features in descending order of importance. The `np.argsort(feature_importance_scores)[-1]` function achieves this by returning the indices that would sort the importance scores, and then reversing the order to get the indices of the most important features first. The `ranked_feature_names` variable stores the names of the features in this importance-based order.
- **Pipeline Construction for Feature Selection:** A Pipeline was created to integrate preprocessing (standardization using `StandardScaler`) and model training (`best_mlp`). This pipeline was designed to be used with a varying number of top-ranked features.
- **Experimenting with Different Numbers of Features:** A loop was implemented to train and evaluate the `best_mlp` model using an increasing number of the top-ranked features (from 1 up to a maximum of 20 or the total number of features). For each number of features.

We calculated the Mean squared Error (MSE) and R-squared score as metrics to judge the performance of the model given k features. We then plotted these value to see the relationship of the performance of the model given k number of features.

The results of the feature selection experiment showed how the model's performance varied with the number of top-ranked features used:

- Initially, as more of the most important features were added, both the MSE decreased (indicating better prediction accuracy) and the R squared increased (indicating a better fit to the data).
- The output shows that the performance generally improved significantly as the number of features increased, reaching a very low MSE and a very high R squared.

- **Diminishing Returns and High Performance:** After retraining the model multiple times and observing the performance around 8 features, the Test Mean Squared Error (MSE) consistently falls within a range of at most 10,000 to a few hundred. Considering the average Seed Yield per Unit Area (SYUA) is approximately 4388.37, an MSE of 10,000 corresponds to a Root Mean Squared Error (RMSE) of around 100, indicating an average prediction error of roughly 2.28 percent and an estimated accuracy of about 97.7 percent. Conversely, an MSE in the low hundreds (e.g., around 252 from a prior run) yields an RMSE of approximately 15.89, translating to a much lower prediction error of about 0.36 percent and an estimated accuracy of around 99.6 percent. This range of high accuracy (97.7 percent to nearly 99.9 percent) achieved with 8 features, compared to the potentially marginal gains with further feature inclusion and increased model complexity, suggests that 8 features strikes an effective balance between robust predictive power and computational efficiency, representing a practical "elbow point" for feature selection.



0.6 Addressing noisy / missing features

The methods used to address missing and potentially noisy features are implemented within the preprocessing_pipeline that is applied before the feature selection in the final pipeline:

- **Missing Features:** Missing values are handled using the SimpleImputer with the strategy='median'. This step replaces any missing numerical values in the training data (train_X_regr) with the median of the non-missing values for each respective feature. This ensures that the model can be trained and evaluated even if there are missing data points.
- **The StandardScaler** is applied to standardize the features after imputation. This process scales each feature to have a mean of 0 and a standard deviation of 1. Standardization can help in mitigating the impact of noisy features by reducing the influence of outliers. It does this by making extreme values which are usually indicative of noise, come closer to the distribution of other data points, preventing them from significantly influencing the model